

Plano de Testes - Challenge 02 | ServeRest

Plano de Testes: ServeRest

Autor: Bárbara Padilha

Swagger: <https://compassuol.serverest.dev/#>

1. Apresentação

Este documento detalha o planejamento, a execução e os resultados dos testes realizados na API ServeRest, que possui como módulos: **Login, Usuários, Produtos** e Carrinhos. O objetivo é garantir a qualidade e a conformidade da aplicação com a documentação e os requisitos definidos nas **User Stories**, sendo elas **US_001, US_002 e US_003**. Este plano foi elaborado para documentar o processo de testagem, permitindo que os resultados sejam agregados à análise e nas evidências.

2. Objetivo

O objetivo deste plano de testes é a validação das funcionalidades da API ServeRest, identificando possíveis bugs, inconsistências e vulnerabilidades. Através da execução dos cenários de teste, busca-se a garantia que a API se comporte conforme o esperado e conforme documentado, tanto nos cenários de sucesso quanto nos demais cenários sugeridos. E em principal, que os *Acceptance Criteria* estabelecidos sejam validados e executados com sucesso.

3. Escopo

O escopo do trabalho contempla as rotas de Usuários, Login e Produtos, como requisitos de teste obrigatórios, abrangendo operações de criação, listagem, atualização e exclusão de recursos, além de validações específicas relacionadas a autenticação, regras de unicidade e restrições de formato. Por fim, a rota de Carrinhos também foi igualmente testada, ainda que não houvesse uma US atrelada, para garantir a consistência e confiabilidade da API de ponta a ponta.

4. Análise

A análise realizada levou em consideração os critérios de aceite descritos em cada User Story e a documentação disponibilizada via Swagger. Foram indicados, no tópico *Acceptance Criteria*, pontos de atenção importantes, como a necessidade de validar e-mails com formato adequado e de restringir provedores como Gmail e Hotmail na criação de usuários, a obrigatoriedade de senhas respeitarem um intervalo entre no mínimo cinco e no máximo dez caracteres, a garantia de unicidade nos cadastros de

usuários e produtos, além do correto tratamento de tokens de autenticação, incluindo validade e expiração.

A execução dos testes, no entanto, revelou inconsistências entre o comportamento esperado e o comportamento real da aplicação. Estas divergências foram documentadas como pontos de defeitos e melhorias, sendo elas:

- **Violação de Regras de Negócio na US 001:** Foi verificado que o sistema permitia o cadastro de usuários com domínios de e-mail proibidos, como @gmail e @hotmail, contrariando diretamente o critério de aceite. Da mesma forma, as restrições de tamanho da senha não foram aplicadas, permitindo cadastros com menos de 5 ou mais de 10 caracteres. A suíte de regressão do Robot Framework confirmou que o sistema falha em impor o limite de caracteres para a senha, permitindo o cadastro de senhas com menos de 6 caracteres (SCRUM-11) e aceitando e-mails com domínios proibidos (SCRUM-28), violando a regra de segurança da US_001. Estes cenários foram configurados para falhar intencionalmente (Status 201 != 400) no teste, atuando como monitores de bugs.
- **Exposição de Dados na US 003:** Foi constatada a ausência de autenticação obrigatória para as operações de listagem (GET /produtos) e busca por ID (GET /produtos/{_id}). Este comportamento de acesso irrestrito aos dados foi formalmente documentado pelos testes automatizados (SCRUM-18 e SCRUM-19), que esperavam o Status Code 401 Unauthorized (falha de segurança) e, em vez disso, retornaram Status Code 200 (OK). A falha intencional do teste (200 != 401) é o indicador de que o bug persiste, sendo este um risco Crítico de segurança.
- **Inconsistência de Lógica e Retorno de Status Code:** O PUT em IDs de usuários e produtos inexistentes se comportou como esperado (criando um novo recurso), mas o teste de exclusão de produtos inexistentes retornou um status code 200 (OK), indicando um sucesso inexistente. Da mesma forma, foi verificado que a API permite o cadastro de produtos com a quantidade zero, afetando a lógica de estoque e a integridade dos dados. A automação também confirmou o bug de lógica de estoque, onde o cadastro de produtos com quantidade zero é aceito (SCRUM-29), violando a integridade dos dados e o fluxo de negócio da API. O teste foi configurado para falhar (201 != 400) para monitorar a correção deste defeito. Adicionalmente, as regras de unicidade e integridade no módulo Carrinho foram validadas, como a restrição de cadastro duplicado (SCRUM-33) e a validação de produto inexistente (SCRUM-34).

5. Técnicas Aplicadas

A metodologia de teste seguiu uma abordagem híbrida, combinando técnicas de teste tradicionais com automações como facilitadores. Para assegurar a conformidade da API, foram aplicadas as seguintes técnicas:

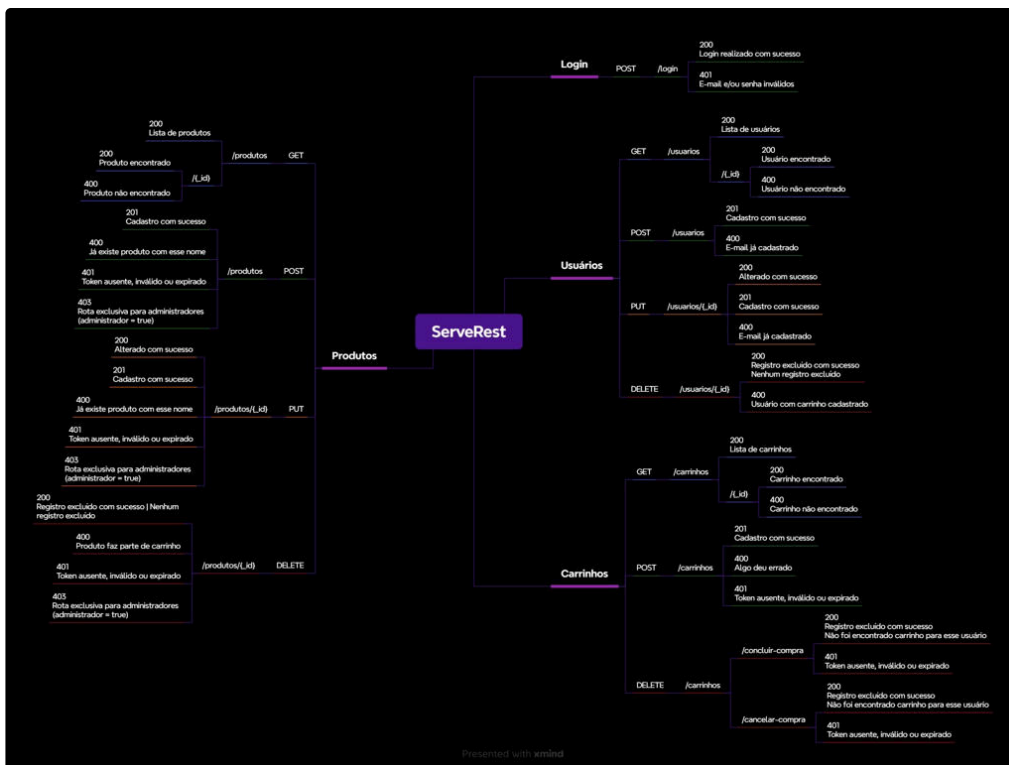
- **Teste Funcional:** Verificação da lógica de negócio e do fluxo de trabalho da API em todas as operações CRUD (Create, Read, Update e Delete). Este teste confirmou se as funcionalidades, como o cadastro de usuários e a criação de produtos, operam conforme os requisitos.
- **Teste de Fronteira (Boundary Value Analysis):** Aplicação de testes nos limites de validação, como o tamanho mínimo (5 caracteres) e máximo (10 caracteres) da senha de usuário. Essa técnica foi necessária para identificar a falha na validação da regra de negócio da US 001.
- **Teste de Regressão:** Através da automação, os cenários foram reexecutados para garantir que novas funcionalidades não introduziram bugs em partes já testadas do sistema.

A totalidade dos cenários de teste foi também automatizada em sua maioria, usando a aba Scripts do Postman. Essa metodologia incluiu:

- **Geração de Dados:** Utilização de **Pre-request scripts** para criar dados de teste únicos, como e-mails e senhas aleatórios, garantindo que os testes sejam otimizados e seja desnecessário o preenchimento manual de determinadas informações, como senhas e e-mails.
- **Validação :** Implementação de scripts na aba Tests para validar o contrato da API (status code, estrutura JSON e response body), validando se o resultado da requisição é o esperado.
- **Gestão de Sessão:** Automação do fluxo de login e da gestão do token de autenticação, o que permite testar fluxos completos de usuário, sem intervenção manual. O armazenamento das informações foram direcionados ao Environment do próprio Postman.

6. Mapa Mental da Aplicação

O mapa mental a seguir representa a estrutura lógica da API, detalhando os módulos, rotas e cenários de teste planejados para cada uma das User Stories.



Mapa mental direcionado a API. Criado pelo Xmind.

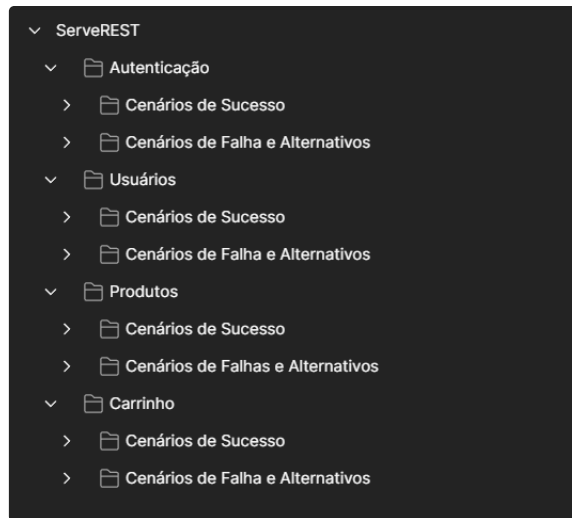
7. Cenários de Teste Planejados e Executados

Os cenários de teste foram organizados em uma Collection Postman, servindo como a principal ferramenta de execução e evidência dos testes. A estrutura da collection é modular e reflete os requisitos das User Stories. As pastas foram estruturadas da seguinte forma:

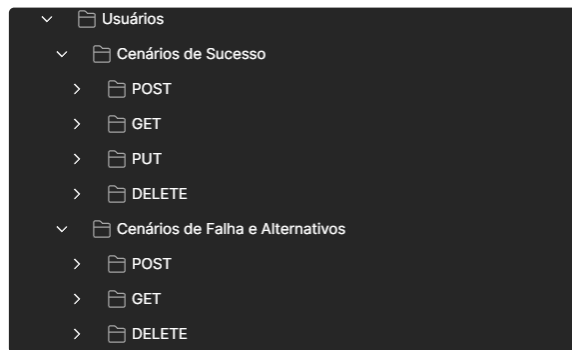
1. ServeRest: Root. Pasta principal.
 - a. Autenticação
 - b. Usuários
 - c. Produtos
 - d. Carrinho

E dentro de cada módulo, temos a seguinte estrutura:

- Autenticação:
 - → Cenários de Sucesso → Rotas da API (GET, POST, PUT, DELETE) divididas em sub-pastas
 - → Cenários de Falhas e Alternativos → Rotas da API (GET, POST, PUT, DELETE) divididas em sub-pastas



Exemplo de Estrutura Modular com as pastas dos cenários



Exemplo de Estrutura Modular com os tipos de requisições

A estrutura segue a mesma lógica para demais pastas como Usuários, Produtos e Carrinhos.

Cada cenário foi nomeado com um prefixo que garante rastreabilidade, sendo representados por ID da User Story seguido da sigla de identificação do teste, podendo ser: **AC** para Acceptance Criteria, ou **ALT** para cenários alternativos. A execução dos testes foi realizada em cada uma das rotas mapeadas, cobrindo os seguintes módulos:

→ **Módulo de Autenticação: US 002**

Os testes neste módulo focaram na validação do fluxo de login.

- **Cenários de Sucesso:** A requisição POST para login foi testada com credenciais válidas, garantindo que o sistema retorna um status code 200 e um token de autenticação, o qual foi automaticamente salvo no ambiente do Postman.
- **Cenários de Falha e Alternativos:** Foram validados os cenários de login com e-mail inexistente e senha inválida. Adicionalmente, cenários alternativos testaram a resiliência da API a campos com formatos inválidos, vazios ou ausentes, garantindo um tratamento de erro consistente.

→ Módulo de Usuários: US 001

Este módulo abrangeu o CRUD completo de usuários. A automação com Pre-request Scripts foi utilizada para gerar dados de teste dinâmicos, garantindo a idempotência e a confiabilidade dos cenários.

- **Cenários de Sucesso:** As operações de POST (cadastrar), GET (listar e buscar por ID), PUT (editar) e DELETE (excluir) foram testadas com sucesso para validar a funcionalidade básica do CRUD.
- **Cenários de Falha e Alternativos:** O foco principal foi nas regras de negócio da US 001. Cenários testaram a não permissão de cadastro com e-mail já existente, e-mails de provedores proibidos (@gmail e @hotmail), e senhas fora do limite de 5 a 10 caracteres. Adicionalmente, foi testado o comportamento da API ao editar um e-mail já em uso por outro usuário e ao tentar buscar/excluir um usuário com um ID inválido ou inexistente.

→ Módulo de Produtos: US 003

Os testes neste módulo validaram o CRUD de produtos, com a particularidade de exigirem autenticação via token.

- **Cenários de Sucesso:** As operações de POST, GET, PUT e DELETE foram executadas com sucesso para um produto existente, utilizando um token válido.
- **Cenários de Falha e Alternativos:** Foram testados cenários críticos, como a tentativa de cadastrar um produto sem token, com nome já existente, com campos ausentes ou com valor de quantidade zero. A execução de testes revelou bugs de segurança críticos, onde as rotas de listagem e busca por ID de produto não exigiam autenticação, o que foi documentado como um defeito.

8. Priorização da Execução

A priorização seguiu a seguinte classificação:

- **Prioridade Alta:** Cenários de validação do *Happy path* e funcionalidades essenciais. Isso incluiu a criação de usuários e produtos, além do fluxo de login, por serem pré-requisitos fundamentais para todas as outras operações. A validação das principais regras de negócio, como a exclusão de usuários e produtos e a autenticação, também foi classificada como alta prioridade devido ao seu impacto na integridade e segurança do sistema.
 - **Prioridade Média:** Cenários de validação de dados e de fluxo secundário. Esta categoria incluiu testes de GET (listagem e busca por ID) e atualizações com dados válidos, que garantem a usabilidade e a consistência do sistema sem serem bloqueadores para o fluxo principal.
 - **Prioridade Baixa:** Cenários de borda e validações mais detalhadas. Cenários alternativos, como o tratamento de dados com formatos inválidos, valores nulos ou casos extremos, foram classificados como baixa prioridade. Embora não impeçam o uso do sistema, a validação desses cenários é importante para atestar a resiliência da API.
-

9. Matriz de Risco

Risco	Probabilidade	Impacto	Nível de Risco	Estratégia de Mitigação
Riscos na Lógica de Negócio (CRUD)	Média	Alto	Alto	Aprofundar a validação das regras de negócio em todas as operações do CRUD. Criar cenários de teste específicos para casos de borda e cenários negativos.
Inconsistência na Validação de Dados de Entrada	Alta (em vigor)	Médio	Médio	Implementar validações de back-end em todas as rotas e desenvolver testes automatizados de validação que cobrem formatação e valores incorretos ou nulos.
Riscos de Segurança	Alta (em vigor)	Crítico	Crítico	Verificar o card de bug

				criado pelo QA no Jira e validar todo o fluxo de autenticação das rotas GET do /produtos, para garantir uma maior segurança dos dados.
Inconsistências em Retornos de API	Média (em vigor)	Baixo	Baixo	Padronizar os códigos de status e as mensagens de resposta da API para que reflitam com precisão o resultado da operação. Sugerir o uso de status code como 400 Bad Request para erros na requisição e 404 Not Found para recursos inexistentes.

10. Cobertura de Testes

A cobertura de testes na API ServeRest foi planejada de forma ampla, garantindo não apenas a validação das funcionalidades descritas nas User Stories, mas também explorando cenários alternativos e situações não documentadas no Swagger da aplicação ou na descrição do Challenge. O objetivo principal foi assegurar que a API atendesse às regras de negócio e aos critérios de aceitação definidos, ao mesmo tempo em que se identificassem vulnerabilidades, inconsistências e oportunidades de melhoria.

10.1. Cobertura por Verbos HTTP (CRUD):

Todos os métodos disponíveis foram verificados, com a execução de operações de Create (POST), Read (GET), Update (PUT) e Delete (DELETE) aplicadas às rotas de Usuários e Produtos. Essa abordagem garantiu a validação completa do ciclo de vida dos recursos, contemplando tanto o fluxo esperado quanto situações de exceção. O foco não esteve apenas na verificação de respostas positivas, mas também na análise de como a API se comporta diante de entradas inválidas, recursos inexistentes ou operações inconsistentes.

10.2. Cobertura por Regras de Negócio e Casos de Uso:

Todos os verbos fundamentais da arquitetura REST (Create, Read, Update e Delete) foram validados nas principais rotas da aplicação, como Usuários e Produtos. Essa cobertura permitiu comprovar que as operações básicas estão implementadas e respondem corretamente. Além disso, foi possível identificar oportunidades de melhoria, como o tratamento inadequado de alguns **status codes** em operações de exclusão, que retornavam 200 OK independentemente do sucesso ou falha da requisição. Esse tipo de verificação garante que a API mantenha consistência em seu comportamento, algo essencial para aplicações que dependem dela.

10.3. Cobertura por Cenários de Sucesso, Falha e Alternativos:

Além dos *Happy Paths*, houve também a cobertura de demais cenários, incluindo-se os casos alternativos.

- **Cenários de Sucesso:** Validação de fluxos onde a API se comporta como esperado (Ex: Login com sucesso, criação de produto).
- **Cenários de Falha:** Verificação de que a API retorna erros apropriados em situações como credenciais incorretas ou dados duplicados.
- **Cenários Alternativos e de Borda:** Análise de como a API lida com valores extremos, nulos, vazios ou em formatos inválidos. Dentro deste tópico houve descoberta de bugs importantes, como a permissão de cadastrar produtos com quantidade zero que prejudicam diretamente a lógica de negócios e de estoque.

10.4. Cobertura de Segurança e Autenticação:

Outro ponto foi a análise e exploração de segurança e autenticação. Os testes abrangeram a geração e o ciclo de vida dos tokens, assegurando que o processo de autenticação funcionasse conforme esperado. Entretanto, a execução dos testes revelou uma vulnerabilidade grave: a possibilidade de listar e buscar produtos sem a necessidade de autenticação. Essa falha vai contra os critérios estabelecidos e representa um risco de exposição de dados.

11. Testes Candidatos a Automação

Todos os cenários de teste mapeados e executados durante o processo foram considerados candidatos à automação e, de fato, automatizados. A automação foi conduzida por meio da Collection do Postman, utilizando scripts nas abas Pre-request e Post-request, garantindo verificações tanto antes da requisição quanto após a execução de fato.

Os principais processos automatizados foram:

- **Validação da API:** Implementação de verificações automáticas para garantir que os status codes (200, 201, 400, 401, entre outros) e a estrutura dos corpos de resposta em JSON estivessem sempre de acordo com o esperado.
- **Gestão Dinâmica de Dados:** Uso de Pre-request Scripts para a criação de dados de teste dinâmicos e únicos, como geração automática de e-mails e senhas. Essa estratégia assegurou que os cenários fossem independentes e não sofressem interferência de execuções anteriores, para facilitar o fluxo de futuros testes também.
- **Teste de Fluxos de Negócio:** Automação do processo de login e armazenamento do token de autenticação em variáveis de ambiente. Isso viabilizou a execução automatizada das rotas que dependem de autenticação, garantindo a validação de fluxos de negócio de ponta a ponta.

12. Adoção do Robot Framework para Automação de Testes

O processo de testagem da API ServeRest foi conduzido, em primeiro plano, utilizando o Postman como principal ferramenta de execução e automação, com o auxílio de *scripts* em JavaScript para a gestão dinâmica de dados e de sessão. Essa metodologia inicial garantiu a cobertura completa dos cenários e a construção de um cenário de testes capaz de identificar regressões futuras.

Entretanto, para suportar a evolução contínua da aplicação e, sobretudo, para garantir a revalidação constante e automática dos *gaps* críticos identificados – especialmente aqueles relacionados à Violação de Regras de Negócio (US 001) e à Exposição de Dados (US 003), torna-se necessária a adoção de um *framework* de automação mais escalável.

Nesse contexto, é proposto a migração para o Robot Framework (RF), utilizando a RequestsLibrary para os testes de API REST. O Robot oferece uma arquitetura que transcende a automação puramente funcional, transformando os casos de teste em uma documentação e facilitando a colaboração e a manutenção do código.

12.1 Adaptação dos Cenários Automatizados

A migração dos cenários de teste do Postman para o Robot Framework foi pensada de forma a preservar a lógica já existente, mas aproveitando melhor os recursos do Robot. O processo envolve mapear cada elemento usado no Postman para seu equivalente dentro do framework, garantindo que a estrutura e a rastreabilidade fossem mantidas.

→ Estrutura e organização dos testes:

No Postman, os cenários são agrupados em pastas de acordo com os módulos ou rotas (por exemplo, Usuários ou Login). No Robot, essa lógica foi adaptada para arquivos `.robot`, cada um representando uma suíte de testes. Assim, módulos principais da API como Usuários, Produtos e Login passaram a ter um arquivo próprio (ex.: `route_users.robot`), que centraliza todos os casos relacionados.

→ Gestão de dados e autenticação:

No Postman, variáveis de ambiente são utilizadas para armazenar informações como o token de autenticação. No Robot, esse papel passa a ser das Test Variables, associadas a uma keyword de login que roda como Setup. Dessa forma, os testes que exigem autenticação já iniciam com a sessão configurada.

Outro ponto de adaptação é os pre-request scripts usados no Postman para gerar dados dinâmicos (como e-mails únicos). No Robot, essa lógica foi reescrita em Python e encapsulada em keywords de utilidade, como a `Gerar Email Único`. Isso facilita a manutenção e garante a criação de dados dinâmicos sempre que necessário.

→ Validação e contrato da API:

As validações que antes ficavam nos scripts da aba Tests no Postman serão traduzidas para asserções próprias do Robot. Códigos de status (200, 201, 400, etc.) e a estrutura do JSON são verificados por meio de comandos como `Should Be Equal As Strings` e `Get Value From Json`. Assim, as checagens de contrato da API seguem de forma clara e integrada aos casos de teste.

→ Rastreabilidade dos cenários:

A padronização dos nomes dos cenários será mantida. No Postman, já existia a prática de nomear cada caso com um prefixo que incluía o ID da user story e o tipo de critério (ex.: `US_001_AC`). No Robot, essa convenção será preservada, garantindo a rastreabilidade entre planejamento e execução dos testes.

12.2 Quantidade de Testes

A quantidade e especificação/título de cada teste mapeado para o Robot está especificado a seguir:

Challenge 02 — (25 tickets)

1. **SCRUM-25** [ROTA/carrinho] Listar carrinhos sem token

2. **SCRUM-19** [ROTA/produtos] Listar produtos (sem token)
3. **SCRUM-11** [ROTA/usuarios] Cadastrar usuário com senha menor que 5 caracteres
4. **SCRUM-14** [ROTA/usuarios] Cadastrar usuário com senha > 10 caracteres
5. **SCRUM-28** [ROTA/usuarios] Cadastrar usuário com email com @gmail.com
6. **SCRUM-18** [ROTA/produtos] Cadastrar Produto com quantidade 0
7. **SCRUM-8** [ROTA/usuarios] Cadastrar usuário válido
8. **SCRUM-10** [ROTA/usuarios] Cadastrar usuário com e-mail inválido
9. **SCRUM-26** [ROTA/produtos] Excluir produto contido em um carrinho
10. **SCRUM-20** [ROTA/produtos] Editar produto pelo ID (com token)
11. **SCRUM-21** [ROTA/produtos] Deletar produto pelo ID (com token)
12. **SCRUM-17** [ROTA/produtos] Cadastrar produto com token de autenticação válido
13. **SCRUM-15** [ROTA/produtos] Buscar produto por ID (sem token)
14. **SCRUM-30** [ROTA/produtos] Cadastrar produto com nome já existente
15. **SCRUM-33** [ROTA/carrinho] Cadastrar carrinho duplicado
16. **SCRUM-13** [AUTH] Login com e-mail inválido (não cadastrado)
17. **SCRUM-23** [ROTA/produtos] Editar produto sem token de autenticação
18. **SCRUM-34** [ROTA/carrinho] Excluir carrinho inexistente
19. **SCRUM-24** [ROTA/carrinho] Excluir carrinho
20. **SCRUM-32** [ROTA/carrinhos] Cadastrar carrinho sem token
21. **SCRUM-22** [ROTA/carrinho] Cadastrar carrinho com produto inexistente
22. **SCRUM-16** [AUTH] Login com senha inválida
23. **SCRUM-27** [ROTA/carrinho] Editar carrinho
24. **SCRUM-31** [AUTH] Login válido
25. **SCRUM-12** [ROTA/usuarios] Cadastrar usuário com e-mail já existente

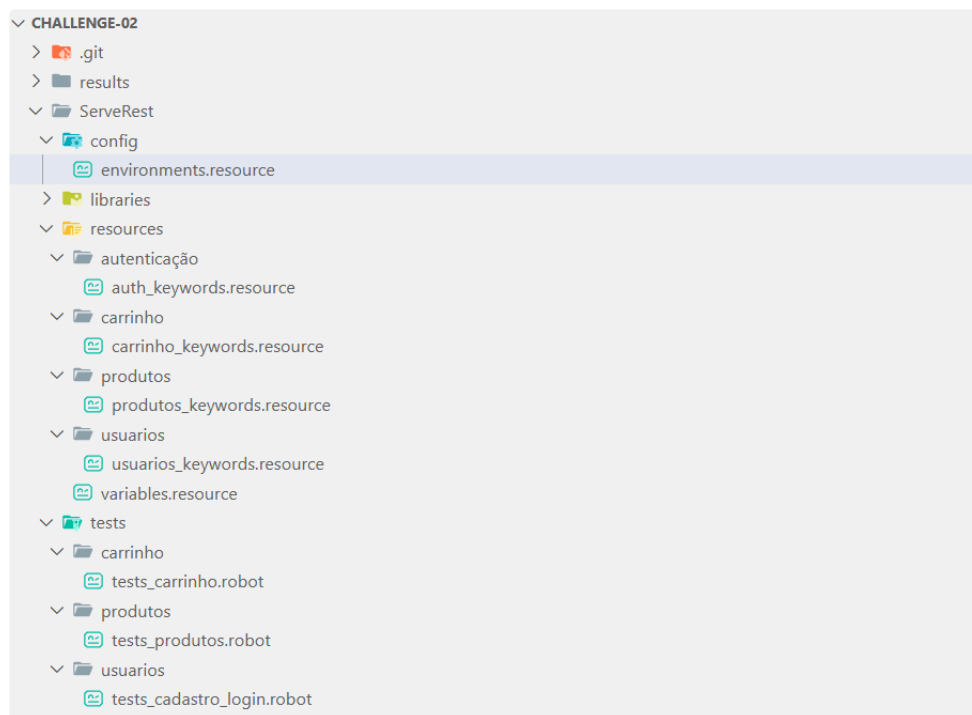
13. Execução dos Testes no Robot Framework

Os testes selecionados para serem contemplados no Robot Framework foram aqueles, em principal, candidatos à automação, e também os testes que validam o fluxo crítico de navegação da API por cada módulo. Dos **60 testes** executados no Postman, sendo manuais e automatizados, **25** deles foram migrados para um refinamento e maior abordagem pelo Robot, garantindo também a confirmação da violação das regras de negócio da ServeRest, além do sucesso na execução dos Happy Paths, como esperado.

A migração e execução confirmaram a estabilidade dos Happy Paths e garantiram o isolamento completo dos cenários negativos. O design do Robot utilizou variáveis de sessão (`${TOKEN_SUITE}`), `${PRODUTO_ID}`) e lógica de Setup/Teardown (criação e exclusão de recursos) para garantir que cada

teste seja independente e idempotente, resolvendo os problemas de colisões de estado (e-mail já existente ou carrinho duplicado) que ocorriam nas execuções sequenciais.

Priorizou-se na estruturação do ambiente para esses testes a mesma modularidade que foi aplicada ao Postman, e mapeada no tópico anterior, contudo, algumas mudanças necessárias foram implementadas para melhorar a rastreabilidade e facilitar manutenção e implementação de novas features futuras. Uma das mudanças notadas e implementadas foi a divisão das subpastas, que antes eram mapeadas por cenário - Sucesso ou Falha/Alternativo - e hoje foram subdivididos em resources, config, tests e também mapeado por rotas (Usuário, Produtos, Autenticação e Carrinho). Observa-se abaixo a arquitetura do repositório:



Arquitetura de Repositório para o Challenge 02.

Todos os 25 testes planejados foram executados, sendo eles **18 com sucesso** e **7 com falhas** já esperadas, pelo mapeamento anterior dado nos testes via Postman.

Tests

| FAIL |

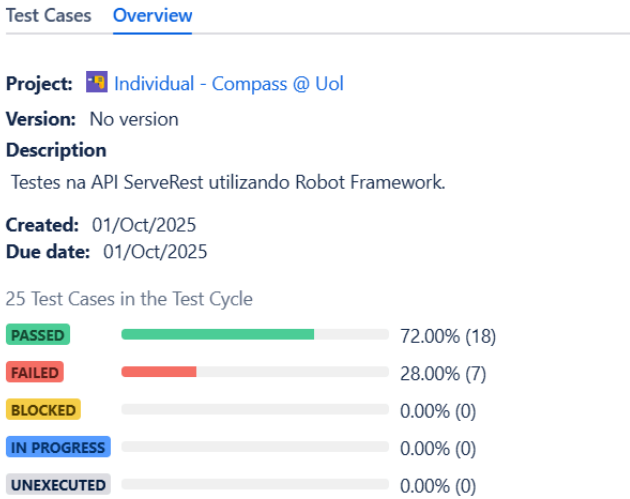
25 tests, 18 passed, 7 failed

13.1 Uso do QALity para Rastreamento e Gestão

A gestão do ciclo de vida dos testes e rastreabilidade (Requirements Traceability) foi realizada via QALity no Jira. O Test Cycle (Ciclo de Testes) foi utilizado como caminho para os resultados de execução da suíte Robot.

1. **Vínculo com o Código:** Cada Test Case (Task) no QALity (Ex: SCRUM-18), em principal os bugs, foi detalhado com os passos "Dado/Quando/Então" da abordagem BDD.

2. **Rastreamento de Falhas (Monitores de Bug):** Os 7 cenários que resultaram em falha (Status: FAIL) no relatório do Robot foram marcados como FAIL no Test Cycle do QAlity, documentando que a API violou a regra de negócio e falhou de acordo com as User Stories.
3. **Conclusão do Ciclo:** A conclusão do ciclo de testes atesta que a regressão crítica está estável (18 PASSES) e que as vulnerabilidades críticas estão sob monitoramento (7 FAILs), concluindo o requisito de qualidade contínua do Challenge 03.



Overview do Test Cycle realizado no QAlity

Houve também a documentação formalizada e padronizada dos registros de bugs, evidenciando os logs gerados pelo próprio Robot e também um template de report para garantir a boa descrição. Além disso, o passo a passo para executar cada bug e o resultado esperado foram armazenados nas tasks do Jira, para melhor rastreabilidade e interpretação dos resultados.